

LLM explainability

Alessandra Ciciarelli

December 2025

1 Fine tuning dataset

To fine-tune LLaMA 3 8B for answer generation and provenance explanation, we construct a dataset where each training example is represented as a structured `<input,output>` pair. The input consists of a natural language question and its corresponding SQL query, while the output is a JSON object containing both the query answers and their provenance, expressed as witness sets. Each answer is explicitly paired with a single witness set.

An example of a training instance is reported below:

```
{
  `instruction`: `Return ONLY valid JSON (no extra text).Output MUST be a JSON array.
  Each array element MUST be an object with EXACTLY these keys:
  - result: an object representing one output tuple
  - provenance: a Why[X] provenance expression for that tuple, where each provenance
  identifier
  is a string formatted as ``<table_name>_<row_number>'' (e.g., ``standings_35''').
  The provenance field MUST be a list of lists of provenance identifiers.
  Each inner list contains the provenance identifiers that together produce the result
  tuple.JSON schema:
  [{``result``: \{...\}, ``provenance``: [[``t1``,``t2``], [``t3``], ...]}, ...]
  Do NOT output SQL, explanations, markdown, or additional keys.If there are no results,
  return [].`,
  `input`: {
    `question`: `What are the dates and points for each constructor standing, along with
    their corresponding
    subsequent standings, for the 28 most recent races?`
  },
  `output`: [
    {
      `result`: {
        `raceid`: `21`,
        `points`: `3`,
        `constructorresultsid`: `38`
      },
      `provenance`: [
        [ `constructor_results_38` ]
      ]
    }
  ]
}
```

2 Curriculum Learning

Curriculum learning (CL) is a training strategy that trains a model from easier data to harder data, imitating learning order in human curricula. In the paper *A Survey on Curriculum Learning?*, training examples are ordered according to an explicit difficulty measure, such as the number of joins, self-joins, and output tuples. The model is first trained on simpler instances and progressively exposed to more complex queries, allowing it to learn the rigid output format and the semantics of witness sets before handling harder relational patterns.

Algorithm 1: Baby Step Training Scheduler?

Input: D : training dataset; C : difficulty measurer
Output: M^* : trained model
 $D' \leftarrow \text{sort}(D, C)$;
 $\{D_1, D_2, \dots, D_k\} \leftarrow D'$ such that $C(d_a) < C(d_b)$, $d_a \in D_i$, $d_b \in D_j$, $\forall i < j$;
 $D_{\text{train}} \leftarrow \emptyset$;
for $s = 1$ **to** k **do**
 $D_{\text{train}} \leftarrow D_{\text{train}} \cup D_s$;
 while *not converged for p epochs* **do**
 train(M , D_{train});

3 17/12/2025 meeting

Current Status

- Three datasets have been generated for model fine-tuning, namely *RELF1*, *RELSTACK*, and *TPC-H*, each consisting of 2000 SQL queries produced using *SQLSmith*.
- For each query, the result set and a corresponding *why*-provenance are available; however, the provenance information is currently represented as unstructured strings, limiting its usability for systematic processing and learning.
- The generated queries are not yet categorized according to their structural or semantic complexity (e.g., joins, aggregates, or set operations), preventing controlled evaluation across different query classes.
- The query generator currently excludes negation operators, as the *EXCEPT* clause in *SQLSmith* has been disabled.
- The fine-tuning datasets do not yet include the full relational context (i.e., all relevant tuples and attributes) required to answer each query in isolation.

Planned Work and Motivation

- **Dataset size characterization.** We will compute the effective size of each dataset by considering not only the number of queries, but also the average number of tuples in query results and the average size of the *why*-provenance. This will provide a more realistic estimate of dataset complexity and learning signal, enabling fairer comparisons across datasets and query types.
- **Structured provenance representation.** The *why*-provenance will be converted from a string-based format to a structured JSON representation based on lists of lists. This structured encoding is essential to support automated manipulation, facilitate supervision during fine-tuning, and enable future provenance-based reasoning tasks.
- **Query categorization by structural features.** The query generation and sampling process will be extended to annotate queries according to key characteristics, including:
 - number of joins,
 - presence of set operations (UNION, INTERSECTION),

- use of aggregate functions (binary indicator and count),
- presence of negation.

This categorization will allow controlled experimentation and evaluation, as well as targeted training on queries of increasing structural complexity.

- **Support for negation.** To include negation patterns, the `EXCEPT` clause in SQLSmith will be re-enabled. Negation is a fundamental relational operator and is known to increase query difficulty; incorporating it is therefore crucial for assessing the robustness of the proposed approach.
- **Specialized fine-tuning by query type.** Separate fine-tuning runs will be performed for different query categories (e.g., single-join, multi-join, aggregate queries), with model checkpoints stored after each training phase. This will enable an analysis of how query complexity affects learning dynamics and model performance.
- **Context-complete fine-tuning data.** Each fine-tuning example will be augmented with the full relational context required to answer the query, including all relevant rows and attributes. Providing complete context is necessary to prevent implicit information leakage and to ensure that the model’s predictions are grounded solely in the supplied data.
- **Query quality refinement.** Duplicate attributes in the `SELECT` clause will be removed during query generation to improve query quality and avoid introducing redundant or misleading supervision signals.
- **Model fine-tuning.** Once the datasets are finalized, supervised fine-tuning will be performed to adapt the model to structured query answering with provenance-aware explanations.
- **Model distillation.** After fine-tuning, knowledge distillation techniques will be explored to transfer the acquired capabilities to smaller or more efficient models, improving deployability while preserving performance.
- **Provenance-to-Natural-Language translation.** Finally, we will design and implement a Prov-to-NL translation component aimed at generating faithful and interpretable natural language explanations from structured provenance, improving the transparency and usability of the system.

Updates on work

- **Metadata added for query type classification** The metadata for classifying query based on their complexity have been added. For example:

```
-- meta {`num_joins`:1,`num_aggregates`:0,
`has_union`:true,
`has_intersect`:false,
`has_negation`:false}

-- meta {`num_joins`:2,
`num_aggregates`:0,
`has_union`:true,
`has_intersect`:false,
`has_negation`:false}
```

- **Set operations in SQLSmith.** Set operations were enabled in the query generator to increase query diversity. In particular, `UNION` and `EXCEPT` are now supported, while `INTERSECT` remains disabled due to limited support in provsql.
- **Dataset statistics.** Dataset statistics are now computed from the final JSONL files, including the number of successful and failed queries, the total number of answer tuples, and detailed provenance size. For example, for the dataset *relf1*:

```

`db`: `relf1`,
`input_sql`: `set_queries/queries_relf1_mixedwithset.sql`,
`output_jsonl`: `queries_with_prov/relf1_mixedwithset_prov.jsonl`,
`generated_at`: `2026-01-07T12:34:56`,
`total_lines_in_jsonl`: 2000,
`ok_queries`: 1999,
`failed_queries`: 1,
`total_answer_tuples`: 47785,
`total_why_sets`: 54752,
`total_why_items`: 83820,
`avg_answers_per_query`: 23.904452226113058,
`avg_why_items_per_query`: 41.93096548274137,
`avg_why_items_per_tuple`: 1.754106937323428,
`effective_size_proxy`: 133604

```

- **Structured provenance representation.** The *why*-provenance is no longer stored as an unstructured string, but is represented as a normalized list of lists in JSON, where each inner list corresponds to a witness set. This format facilitates downstream processing and model supervision.
- **SQL to NL** The sql to nl json files have been produced for the three datasets: relf1, relstack and tpch. The format of each row is:

```

{
  `id`: 1999,
  `meta`: {
    `num_joins`: 0,
    `num_aggregates`: 0,
    `has_union`: false,
    `has_intersect`: false,
    `has_negation`: false
  },
  `sql`: <sql_query>,
  `nl`: <generated_nl_query>
},

```

4 Meeting 07/01/2026

Meeting Summary (Prompt and Dataset Refinement)

- The fine-tuning prompt will be revised to exclude SQL from the input and to provide more precise and correct instructions to the model.
- A separate set of queries will be created for validation, ensuring they are different from those used during training.
- Instead of multiple datasets, a single dataset will be used for the fine tuning, enriched with explicit annotations capturing query complexity. This because we need one model able to handle different type of queries.
- Negative pairs will be added to the fine-tuning dataset by perturbing positive examples, e.g., by adding or removing tuples or sets, to improve the model's discriminative capabilities.

5 Observation and limitations

During a manual inspection of the generated SQL queries, their automatically generated natural language (NL) questions, and the corresponding outputs with provenance produced by ProvSQL, several limitations emerged that can directly affect the quality of the fine-tuning dataset.

In our setup, the output of the execution of the SQL queries, together with the provenance, is always correct with respect to the SQL semantics (we are using ProvSQL to generate it). However, the NL questions are automatically generated using an LLM and represent the weakest component of the pipeline, introducing a significant source of noise. This asymmetry is intrinsic to the generation process and reflects a realistic scenario in which user queries may be ambiguous or imprecise.

A major source of ambiguity arises from the systematic use of `LIMIT` clauses without `ORDER BY`. While `LIMIT` is necessary to keep query results and provenance expressions manageable, it induces non-deterministic semantics, as the returned tuples represent an arbitrary subset rather than ranked or semantically “top” results. This often leads NL translations to hallucinate notions such as “top”, “best”, or “most recent”, which are not justified by the SQL query. Introducing `ORDER BY` is not an easy solution, as the query generator has no access to data distribution or domain semantics, and the language model does not observe the ordered context that would justify such constraints.

Additional issues come from set operations such as `UNION`, which remove duplicates by definition and therefore make the final cardinality of the result unpredictable, even when subqueries are bounded by `LIMIT`. Furthermore, the SQLSmith-based query generator frequently produces syntactically valid but semantically weak predicates (e.g., `IS NOT NULL` on primary-key-like attributes), which are difficult to translate faithfully into NL.

Given these considerations, we adopt a ‘curation’ strategy.

- To mitigate the limitations introduced by `LIMIT`, we propose a data curation strategy based on provenance-driven database recomposition. This approach aims to construct a smaller but representative dataset that allows queries to be executed without `LIMIT`, while keeping outputs and provenance manageable and semantically consistent.

Provenance-driven dataset recomposition The following procedure describes the proposed workflow for curating a database using provenance information, enabling the removal of the `LIMIT` clause in a second generation of the dataset.

Algorithm 2: Provenance-based Dataset Curation

Input: Original database D , query generator G

Output: Curated database D' , updated query results and provenance

Generate a set of SQL queries $Q = \{q_1, \dots, q_n\}$ from D using G , each including a `LIMIT` clause

foreach $q_i \in Q$ **do**

 | Execute q_i on D using ProvSQL, obtaining result R_i and provenance P_i

Initialize an empty set of tuples $T_{\text{relevant}} \leftarrow \emptyset$

Extract all tuples referenced at least once in any provenance expression P_i and add them to T_{relevant}

Construct a reduced database D' containing only tuples in T_{relevant}

Re-execute the same set of queries Q on D' after removing all `LIMIT` clauses, collecting new results and provenance

return D' , *updated results, and provenance*

6 Data Pipeline

The construction process merges three primary sources of information:

1. **SQL queries:** SQL queries generated using a customized version of the fuzzy random generator sql smith. This has been adapted to produce queries more adapted for our task and for the execution with the provsql tool.

2. **Provenance Data:** SQL queries executed with provenance computation (using `provsql`), providing the ground truth tuples and their corresponding why provenance sets.
3. **Natural Language (NL) Queries:** NL descriptions of the SQL queries generated using llama3.3-latest : 70.6B.
4. **Context Data:** The actual rows from the source database tables required to answer the query (extracted from the provenance expressions).

7 Negative Samples Generation (DPO)

A critical component of this dataset is the generation of **rejected samples** for Direct Preference Optimization (DPO). Unlike standard SFT (Supervised Fine-Tuning) which only uses correct examples, DPO requires pairs of (chosen, rejected) outputs to teach the model to distinguish between valid and invalid provenance.

7.1 Mutation Strategies

The negative samples are generated by applying a single, controlled mutation to the correct provenance of a randomly selected tuple in the output. We implement five mutation modes to cover common errors:

- **Add Token (Witness Set Extension):** Adds a random row identifier from the context to an existing witness set. This teaches the model not to include irrelevant rows.
Example: `[["circuits_2","races_19"]] -> ["circuits_2","races_19","drivers_11"]`
- **Remove Token (Witness Set Reduction):** Reduces a witness set to a single token. This helps the model understand that multiple rows are often simultaneously necessary (conjunction). Example: `[["circuits_2","races_19"]] -> ["circuits_2"]`
- **Add Set (False Alternative):** Adds a new singleton witness set. This creates a "false alternative" explanation for the result.
Example: `[["circuits_2","races_19"]] -> ["circuits_2","races_19"],["drivers_8"]`
- **Remove Set (Missing Explanation):** Removes one of the witness sets (only if multiple exist). This teaches the model that provenance must be complete.
Example: `[["circuits_2","races_19"],["drivers_8"]] -> ["circuits_2","races_19"]`
- **Replace Token (Substitution):** Replaces a row identifier with another one from the same table. This maintains the schema validity but invalidates the provenance.
Example: `["constructor_standings_33","races_21"] -> ["constructor_standings_34","races_21"]`

7.2 Improving the Construction of t Fine-Tuning Dataset with Negative Pairs

Input files:

- **Provenance-enriched query results (-prov-jsonl):** A JSONL file containing SQL queries, their execution results, and associated Why-provenance information.
- **Natural language queries (-nl-file):** A JSON or JSONL file mapping query identifiers to their corresponding natural language questions.

- **Context data** (-context-jsonl): A JSONL file containing the table rows used as evidence, indexed by query identifier.

Each query is identified by an integer ID, which is used to align SQL, natural language, and context data.

7.2.1 Preprocessing and Filtering

For each candidate query, the script performs the following preprocessing steps:

1. **SQL cleaning:** SQL comments and provenance-specific function calls are removed to avoid noise introduction into the prompt.
2. **Context cleaning:** Internal provenance metadata (e.g., `provsql` fields) are removed from the context rows, leaving only raw attribute values.
3. **Metadata extraction:** Query-level metadata (number of joins, aggregates, set operators, and negation) are parsed from structured comments embedded in the SQL.

Queries are discarded if any of the following conditions hold:

- Missing metadata
- Missing natural language question
- Missing context data
- Empty result set

7.2.2 Query Categorization

Each remaining query is assigned to one of five categories based on its metadata:

- **easy_no_join:** no joins, no aggregates, no set operators
- **medium_join:** one or more joins, no aggregates
- **has_aggr:** aggregate queries
- **has_union:** queries containing UNION only
- **has_all:** queries containing multiple advanced operators (UNION, EXCEPT)

This categorization enables balanced dataset construction and categorized analysis during training and evaluation.

7.2.3 Prompt Construction

For each selected query, a prompt is built using a fixed instruction template. The prompt contains:

- A strict instruction requiring *only valid JSON output*
- The SQL query (used as the question)
- The cleaned context data, serialized as JSON

The expected output format is a JSON array of objects, each containing:

- **result:** one output tuple
- **provenance:** a Why[X] provenance expression represented as a list of lists.

7.2.4 Generation of Negative Outputs

To enable preference-based fine-tuning, the script generates a *rejected* (negative) output for each correct (*chosen*) output.

Negative outputs are produced by applying multiple randomized edits to the provenance annotations while keeping the result tuples unchanged. The edits are designed to mimic realistic model errors observed in practice. Examples include:

- Replacing provenance tokens with identifiers from the same or different tables
- Adding or removing tokens from a witness set
- Adding or removing entire witness sets
- Duplicating table prefixes or row indices (e.g., `nation_1` $\rightarrow$ `nation_nation_1`)
- Flattening multiple singleton witness sets into a single incorrect set
- Rarely, merging the provenance of two distinct result tuples

Each negative example typically applies multiple edits across different tuples, increasing its difficulty and diversity.

The applied corruption strategy is recorded using a `negative_mode` label for later analysis.

7.2.5 Output Format

For each category, the script produces a JSONL file where each line contains:

- `id`: query identifier
- `prompt`: the full model input prompt
- `chosen`: the correct output (JSON-serialized)
- `rejected`: the corrupted output (JSON-serialized)
- `negative_mode`: description of the applied corruption

Additionally, a global statistics file is generated, reporting:

- Number of selected queries per category
- Distribution of query structural types
- Frequency of each negative corruption mode

7.2.6 Purpose and Benefits

The resulting dataset is directly usable for DPO-style fine-tuning and is particularly suited for provenance-aware question answering systems over relational data. It can also be adapted for standard supervised fine-tuning by ignoring the `rejected` field, providing a rich source of provenance-annotated examples for various training paradigms.

8 Adding bad format records

To further enhance the model’s reliability, we incorporate a second category of negative samples focused on **json integrity** and **output format**. While the primary dataset focuses on logical provenance errors, this augmentation step teaches the model to resist common LLM errors, such as adding conversational sentences or deviating from the specified JSON structure.

8.1 Methodology

We employ an LLM-in-the-loop approach (using `Ollama`) to generate "bad-format" rejected samples. By appending adversarial suffixes to the original prompts, we force the generator to produce outputs that violate the schema in specific, weighted ways:

Attack Category	Weight	Description
<code>extra_keys</code>	12.5%	Adds unauthorized keys (e.g., <code>"explain"</code>) to the objects.
<code>wrong_key_name</code>	12.5%	Renames <code>"provenance"</code> to <code>"prov"</code> , testing key-name precision.
<code>wrong_schema</code>	12.5%	Flattens the provenance list or converts it to a single string.
<code>bad_id_format</code>	12.5%	Uses incorrect separators (e.g., <code>table-id</code> vs <code>table_id</code>).
<code>invalid_json</code>	37.5%	Forces syntax errors like unclosed brackets or missing commas.

Table 1: Adversarial strategies for format-based rejected samples.

8.2 Adversarial Filtering Logic

The generation process follows a strict validation loop to ensure the "rejected" sample is truly broken before it is added to the dataset:

1. **JSON Extraction:** The script attempts to isolate a JSON array from the LLM’s raw response, effectively removing any conversational prefixes.
2. **Schema Validation:** The isolated text is passed through a validator (`is_valid_output_schema`) which checks for:
 - Valid JSON syntax.
 - Correct top-level type (Array).
 - Exact key matching (`result` and `provenance`).
 - Regex-based validation of provenance identifiers (`<table_name>_<row_number>`).
3. **Negative Selection:** A sample is only accepted if it **fails** one of these checks, ensuring that the DPO "rejected" field is objectively incorrect according to the task instructions. At the end 100 samples have been added for each dataset (300 in total).

9 Dataset Partitioning and Stratification

The final stage of the pipeline involves shuffling and splitting the merged dataset into training, validation, and test sets. To ensure that the model generalizes well across different types of queries and provenance from different data source, we implement a **splitting strategy**.

9.1 Tag-Based Stratification

Each record in the dataset is identified by a unique ID that includes a "tag" suffix, referring to the source dataset (e.g., `101_t`, `102_rs`). The splitting script uses these tags to maintain the distribution of samples:

1. **Grouping:** Data is grouped by its categorical tag.

2. **Local Shuffling:** Within each group, samples are shuffled using a fixed seed (42) to ensure reproducibility.
3. **Proportional Allocation:** Samples are allocated to the *Train*, *Validation*, and *Test* sets based on predefined ratios (default: 85% / 10% / 5%).
4. **Global Re-shuffling:** Once the sets are formed, they are shuffled globally to prevent the model from learning biases based on the sequence of query types.

10 Cleaning Dataset

Before proceeding to training, the dataset is cleaned and adapted to the template for the fine tuning.

10.1 Token-Based Filtering

We use the `Meta-Llama-3-8B-Instruct` tokenizer to calculate the exact length of each prompt-response pair. This is critical for preventing Out-Of-Memory (OOM) errors during fine-tuning.

- **Prompt Capping:** Examples with a prompt exceeding 2048 tokens are discarded.
- **Pair Capping:** The total length of the (Prompt + Chosen) or (Prompt + Rejected) sequence is capped at 4096 tokens.
- **Key Dropping:** To further reduce token counts, irrelevant database metadata (e.g., `comment`, `text`) are recursively removed from the JSON context without affecting the logical integrity of the query.

11 Format Preparation for Llama 3 Fine-Tuning

The final step converts the raw DPO pairs into the specific chat format required by the Llama 3 instruction-tuned architecture.

11.1 Chat Templating and EOT Tokens

Listing 1: Example of a Formatted DPO Record

```
{
  "prompt": "<|begin_of_text|><|start_header_id|>user<|end_header_id|>\n\nReturn ONLY valid",
  "chosen": "[{\"result\": {...}, \"provenance\": [...]}] <|eot_id|>",
  "rejected": "[{\"result\": {...}, \"provenance\": \"bad_string\"}] <|eot_id|>"
}
```

11.2 LoRA: Efficient Fine-Tuning

Due to the size of the Llama-3-8B model, we utilize **LoRA** to perform parameter-efficient fine-tuning. This approach injects trainable low-rank matrices into the Transformer layers, significantly reducing VRAM requirements while maintaining high performance.

11.3 Hyperparameter Selection and Stability

The training configuration is optimized for stability and performance on modern hardware (e.g., NVIDIA H100). The key parameters are summarized below:

Parameter	Value	Rationale
<code>beta</code>	0.1	Controls the strength of the KL-penalty. 0.1 is standard for stability.
<code>learning_rate</code>	2×10^{-6}	A conservative rate to prevent catastrophic forgetting.
<code>bf16</code>	True	Utilizes Brain Floating Point for higher dynamic range on H100/A100.
<code>lora_r</code>	16	Rank of the update matrices; balances capacity and memory.
<code>lora_alpha</code>	32	Scaling factor for LoRA updates (typically $2 \times r$).
<code>max_length</code>	4096	Full context window for complex schema + provenance output.

Table 2: DPO Training Hyperparameters.

11.4 Training Implementation

The script utilizes the `DPOTrainer` from the *TRL* library. A crucial stability choice is the use of **Bfloat16** precision and the avoidance of 4-bit quantization where memory allows, ensuring higher gradient precision during the preference learning phase.

12 Model Deployment: From Adapters to Ollama

Once training is complete, the process yields a **PEFT Adapter**. This adapter contains the "delta" weights that transform the base Llama-3 model into a provenance-aware assistant.

To facilitate testing and local usage, the adapter can be integrated into **Ollama** using a custom `Modelfile`:

1. **Base:** Use the original Llama-3-8B-Instruct.
2. **Adapter:** Link the path to the trained LoRA weights.
3. **Template:** Ensure the prompt template matches the one used during training.